Abstract

Chapter 1

Introduction

Bipartite Matching is the problem of finding the largest non-neighboring subset of edges of a bipartite graph. It is well-known and well-studied since it can be used to model a variety of real life problems since a bipartite graph can reflect a 'worker-can-fulfill-task' relation and the matching is an assignment of tasks. Online Bipartite Matching is a variant of this problem where the entire graph isn't initially visible and matching decisions have to be made 'on the fly' as the graph is revealed. This model is just as useful, if not more so, in real life scenarios since in the context of many services all tasks aren't known initially.

This simplified model, however, is often a lower bound, not a perfect solution since the assumptions of the model are harsher than reality. For this reason we examine the performance of various algorithms in slightly different models

The contents of the work will be as follows:

Contents

Chapter 2

Preliminaries

for matching M and vertex v matched in M, $M(v)$ is the vertex matched to v in M

Chapter 3

Semi-Online Model

In this section we will introduce the classical smi-online model introduced by Karp et al [KVV90]

In this model the two anticliques of the bipartite graph are divided into offline and online agents. Offline agents are vertices that belong to one anticlique and are always visible. Online agents are the vertices from the other anticlique and arrive in the online fashion. When an online agent arrives edges incident to it are revealed.

The adversary is oblivious as an adaptive adversary is trivial.

Remark 3.0.1. *No algorithm has a competitiveness asymptotically higher than* $\frac{1}{2}$ *against an adaptive adversary*

Proof. Begin with $2n$ offline agents V and $k = 0, X_0 = \emptyset$.
Adversary repeats the following n times: reveals an agent u connected to $V \backslash X_k$.
$k = k + 1$, for some unmatched $v \in V; V_{k+1} = V_k \cup \{v\}$.
Then adversary reveals n agents U' connected to already matched offline agents.
The ultimate size of the matching is n.
In the final graph (figure), all matched vertices V' are connected to all online agents and all unmatched v_i agents are connected to all u_j such that $j \leq i$.
Clearly, it is possible to match v_i to u_i and V' to U', forming a matching of size $2n$

□

Remark 3.0.2. *In the semi-online model, any deterministic algorithm has a competivineness no greater than* $\frac{1}{2}$*, there is a deterministic algorithm with competitiveness of* $\frac{1}{2}$

Algorithm 1 Naive

on Arrival of vertex v: match it with any offline agent if available

Proof. Trivially the competitiveness cannot be greater than $\frac{1}{2}$ as against a deterministic algorithm there is no difference between an adaptive and an oblivious adversary.
To prove that there is a deterministic algorithm with competitiveness of $\frac{1}{2}$ consider the a naive algorithm.
for every $(u, v) \in E(G)$ either u or v is matched. Therefore at least half of all vertices that could be matched are matched, therefore the comptititveness isn't lower than $\frac{1}{2}$ □

The main discovery of the Karp paper is the following algorithm

Algorithm 2 Ranking

on Initialization: create a random permutation of the offline agents σ
on Arrival of vertex v: match it with the first availible offline neighbor according to σ

as well as the following theorems

Theorem 1. *Ranking has asymptotic competitiveness of* $1 - \frac{1}{e}$

Theorem 2. *No algorithm has asymptotic competitiveness higher than* $1 - \frac{1}{e}$

3.1 Ranking Competitiveness

In this section we present a simple proof for Ranking competitiveness (Theorem 1) by [BM08]

Lemma 3.1.1. *Competitiveness is determined by graphs with a perfect matching*

Proof. Let G be a bipartite graph without a perfect matching, with orderings of offline agents σ and of online agents π. Consider G' which is a copy of G without a vertex x and orderings pi', π' induced by $V \backslash \{x\}$.
Observe that $m = \text{Ranking}(G, \pi, \sigma)$ being the matching found by the Ranking algorithm on G with ranking π and arrival order σ is either identical to $m' = \text{Ranking}(G', \pi', \sigma')$ or has one more vertex pair. This is apparent since either:

1. x isn't in m,
2. $x' = M(x)$ isn't matched in m', or
3. x' is matched to y in m'

In the case of 3 there is an alternating path of edges in m' and those that are in m but not in m'. When this path ends in the same partition as x then m is larger than m' by 1
This also leads to the conclusion that if the respective matching sizes are different, they differ by a single alternating path.
Now consider the maximal matching of G and the effect the removal of the vertices outside of it will have on the competitiveness. The size of the maximal matching will (of course) not change, while the Ranking's solution decreases by at most 1 with each removal, implying that this process of reducing G to G_p with a perfect matching didn't increase the competitiveness, meaning that analysis of graphs without perfect matchings is redundant. □

In the following section graphs are assumed to have perfect matchings. Let $M^\star$ be that perfect matching.

Lemma 3.1.2. *Let u online vertex,$v = m^\star(u)$, σ permutation.*
If v isn't matched by $\text{Ranking}(\sigma)$ *then u is matched by* $\text{Ranking}(\sigma)$ *to v' with a lower rank*

Proof. Trivial since u, v are connected, therefore for v to remain unmatched u must've been matched to another vertex, which must've in turn had a lower rank than v □

Lemma 3.1.3. *Let u online vertex,$v = m^\star(u)$, σ permutation, σ_i permutation obtained by removing v and inserting it into sigma at the i-th place.*
If v isn't matched by $\text{Ranking}(\sigma)$ *then for every i* $\text{Ranking}(\sigma_i)$ *matches u with a vertex v_i such that $\sigma_i(v_i) \leq \sigma(v) = t$*

Proof. $m = \text{Ranking}(\sigma), m_i = \text{Ranking}(\sigma_i)$.
Symmetrically to Lemma 3.1.1 m, m_i are either identical or differ by a single alternating path. It is easy to see that offline vertices in that path are of increasing rank in σ_i. Therefore $v_i = m_i(u), v' = m(u), \sigma_i(v_i) \leq \sigma_i(v')$
Using Lemma 3.1.2 we get $\sigma(v') < t$ and since $|\sigma_i(v') - \sigma(v')| \leq 1$ we receive $\sigma_i(v_i) < 1 + t$ equivalent to $\sigma_i(v_i) \leq t$ on integers □

The final lemma required to complete the proof uses this result directly.

Lemma 3.1.4. *The probability over σ that the offline vertex of rank t is matched by* Ranking(σ) *denoted x_t fulfills the following:* $1 - x_t \leq \frac{1}{n} \sum_{1 \leq s \leq t} x_s$

Proof. Let σ be a random permutation of offline vertices. Define σ' with the following random process: choose an offline vertex randomly with equal probability, take it out of σ and insert it into the t-th rank. Let v be the vertex of rank $t, u = m^*(v), R_{t-1}$
online vertices matched by the algorithm to offline vertices of rank $\leq t-1$
probability that v is not matched is $1 - x_t$. Expcted cardinality of R_{t-1} is $\sum_{1 \leq s \leq t-1} x_s$.
Applying Lemma 3.1.3 with $\sigma := \sigma', i$ chosen such that $\sigma'_i = \sigma$: if v isn't matched by Ranking(σ') then u is matched by Ranking(σ) to a vertex v' such that $\sigma(v) \leq t$ implying $u \in R_t$
Conditional on σ, $u \in R_t$ has a probability of $\frac{|R_t|}{n}$ since they are independent.
We now have two random events E, F that fulfill the condition that $E \implies F$.
It is obvious to see that $\mathbb{P}(E) \geq \mathbb{F}$
The lemma follows

□

With Lemma 3.1.4 the Theorem can be proved directly.
G has a perfect matching implies that copetitiveness is inf $\frac{1}{n} \sum_{1 \leq s \leq n} x_s$.
Let $S_t = \sum_{1 \leq s \leq t} x_s$. Lemma 3.1.4 can be rewritten as $S_t(1 + \frac{1}{n}) \geq 1 + S_{t-1}$ and the competitiveness ratio as $\frac{1}{n} S_n$.
Infimum occurs therefore when all inequalities are tight, resulting in $S_t = \sum_{1 \leq s \leq t} (1 - \frac{1}{n+1})^s$
for all t, implying that competitiveness is at least $\frac{1}{n} \sum_{1 \leq s \leq n} (1 - \frac{1}{n+1})^s = 1 - (1 - \frac{1}{n+1})^n \xrightarrow[n \to \infty]{} 1 - \frac{1}{e}$

Thus the theorem is proven

3.2 Ranking Optimality

In this section we present a proof for Ranking Optimality (Theorem 2) by [KVV90]. The main vehicle for this is Yao's lemma with a construction of a class of matching instances

Algorithm 3 RGA

on Arrival of vertex v: match it with a random availible offline neighbor

Definition 3.2.1. *bipartite online matching instance (n, σ) where σ is a permutation of $[n]$ is constructed as follows:*

- *offline vertices are f_i for $i \in [n]$*
- *online vertices are n_j for $j \in [n]$*
- *f_i, n_j are connected iff $\sigma(i) \leq j$*

The collection of all $n!$ (n, σ) instances is M. $U(M)$ is the uniform distribution over M. $M^\star$ is $(n, (1, \ldots, n))$, the original instance without permutation [figure] notice that perfect matching is achived only if f_i is matched with $n_{\sigma(i)}$ for all $i \in [n]$

Lemma 3.2.1. *Let* DGA *be a greedy deterministic algorithm.*
The expected performance of DGA *on a random instance from* $U(M)$ *bounds the expected performance of* RGA *on* $M^\star$

$$\mathop{\mathbb{E}}_{m \sim U(M)}[|\mathrm{DGA}(m)|] = \mathbb{E}[\mathrm{RGA}(M^\star)]$$

Proof. To prove the lemma we first need to prove two claims:

- **Claim 1** for DGA on (n, σ) as well as RGA on $M^\star$ if at time t there are k eligable offline vertices then they are equally likely to be any set of k vertices among the last $n - t + 1$ denominated $F(t)$
- **Claim 2** for each k the probability that there are k eligible offline vertices at time t, denominated p_t, is the same for DGA on (n, σ) as well as RGA on $M^\star$

Both claims are proven with induction on the 'step' of the algorithm - tha amount of online vertices that have already arrived. Note that induction base at step 0 is trivial since all offline vertices are eligible.
Note that a vertex is availible if it can still be matched ie isn't matched and isn't unmatchable

Proof. of Claim 1
for RGA.

$$\Pr[S \text{ Availible at time } t{+}1] = \sum_{v \notin S} \Pr[S \cup \{v\} \text{ Availible at time } t] \Pr[v \text{ picked at time } t]$$

and since probabilities at time t are equal and the probability of choosing all v is either equal or 0 then the condition is maintained
for DGA, since it is deterministic and the algorithm can't read labels, then the subsequent matching is determined by the current availible set, which is uniformly random, implying the next one is also uniformly random. □

Proof. of Claim 2
From Claim 1 we know that, conditional on the amount of eligible vertices, their placement is universal □

The lemma follows directly from Claim 2 since the size of the matching is equal to the amount of steps required for there to be no eligible vertices □

By using Yao's lemma we receive the following: for any algorithm ALG

$$|\mathrm{ALG}(M)| := \min_{m \in M} \mathbb{E}[|\mathrm{ALG}, \mathrm{m}|] \leq \max_{\mathrm{DALG}} \mathop{\mathbb{E}}_{m \sim U(M)} [|\mathrm{DALG}(m)|] = \mathbb{E}[|\mathrm{RGA}(M^\star)|] \tag{3.1}$$

Since (as proven in the previous section) only instances with perfect matchings need be considered in competitiveness analysis, competitiveness of all algorithms is upper bounded by $\mathbb{E}[|\mathrm{RGA}(M^\star)|]/n$

Now, all that is required to prove the following lemma

Lemma 3.2.2. $\mathbb{E}[|\mathrm{RGA}(M^\star)|] \leq n(1 - \frac{1}{e}) + o(1)$

The rest of this section is the proof of this lemma.

Let $\mathrm{rem}(t)$ be the random variable representing the number of remaining online vertices and $\mathrm{avl}(t)$ be the number of offline vertices availible for matching, at the time t. Δf is used conventionally, as the change of $f(t)$ over t.
Δrem is always -1 while Δavl is either -1 or -2. It is only -2 if n_t is being matched and f_t is availible (event A_t^1) and it isn't matched to n_t (event A_t^2) since f_t becomes unmatchable.
Note that rem is essentially a clock ticking backwards since it has value $n-t$

$\Pr[A_t^1|\mathrm{avl}, \mathrm{rem}] = \frac{\mathrm{avl}}{\mathrm{rem}+1}$ since from Claim 1 from Lemma 3.2.1 we know that all avl sized subsets are equally likely to be the availible subset
$\Pr[A_t^2|\mathrm{avl}] = 1 - \frac{1}{\mathrm{avl}} = \frac{\mathrm{avl}-1}{\mathrm{avl}}$ since matched vertex is chosen randomly
Since both events are independent $\Pr[A_t^1 \wedge A_t^2] = \frac{\mathrm{avl}-1}{\mathrm{rem}+1}$
$\mathbb{E}[\Delta\mathrm{avl}] = (-2)\frac{\mathrm{avl}-1}{\mathrm{rem}+1} + (-1)(1 - \frac{\mathrm{avl}-1}{\mathrm{rem}+1})$

finally

$$\mathbb{E}[\Delta\mathrm{avl}] == -1 - \frac{\mathrm{avl}(t) - 1}{\mathrm{rem}(t)} \tag{3.2}$$

implying $\frac{\mathrm{E}[\Delta\mathrm{avl}]}{\mathrm{E}[\Delta\mathrm{rem}]} = 1 + \frac{\mathrm{rem}(t)-1}{\mathrm{avl}(t)}$ since Δrem is always -1
Though this is a discrete Markov Chain, we can approximate it by treating it as a continious function, whose rate of growth is already known

$$\frac{d\mathrm{avl}}{d\mathrm{rem}} = 1 + \frac{\mathrm{avl} - 1}{\mathrm{rem}} \tag{3.3}$$

We also havethe initial conditions, avl = rem = n, meaning we can find an approximating equation by solving *Equation* 3.3

$$\frac{d\text{avl}}{d\text{rem}} = 1 + \frac{\text{avl} - 1}{\text{rem}} \text{ can be rewritten as}$$

$$\frac{d\text{avl}}{d\text{rem}} - \frac{\text{avl}}{\text{rem}} = \frac{\text{rem} - 1}{\text{rem}} \text{ multiply both sides by } m(\text{rem}) = e^{\int -1/\text{rem}\, d\text{rem}}$$

$$\frac{\frac{d\text{avl}}{d\text{rem}}}{\text{rem}} + \frac{d}{d\text{rem}}\left(\frac{1}{\text{rem}}\right)\text{avl} = \frac{\text{rem} - 1}{\text{rem}^2} \text{ apply product rune to LHS}$$

$$\frac{d}{d\text{rem}}\left(\frac{\text{avl}}{\text{rem}}\right) = \frac{rem - 1}{\text{rem}^2} \text{ integrate both sides}$$

$$\int \frac{d}{d\text{rem}}\left(\frac{\text{avl}}{\text{rem}}\right) = \int \frac{\text{rem} - 1}{\text{rem}^2}$$

$$\frac{\text{avl}}{\text{rem}} = \frac{1}{\text{rem}} + \ln \text{rem} + c \text{ apply initial condition avl} = \text{rem} = n$$

$$1 = \frac{1}{n} + \ln n + c \implies$$

$$\text{avl} = 1 + \text{rem}\left(\ln \text{rem} - \ln n + 1 - \frac{1}{n}\right)$$

giving us the final approximation

$$\text{avl} = 1 + \text{rem}\left(\frac{n-1}{n} - \ln \frac{\text{rem}}{n}\right) \tag{3.4}$$

Bearing in mind that the initial Markov chain doesn't perfectly emulate the last step, we substitute rem $= 1$ instead rem $= 0$ and from Equation 3.4 we receive that, when only 1 online vertex has yet to arrive, the number of remaining columns is $n\frac{1}{e} + o(1)$. Bearing in mind that the last arriving vertex will affect the performance by at most 1, the expected size of the final matching is $n\left(1 - \frac{1}{e}\right)$

Chapter 4

Random Arrival

As shown before, relatively simple algorithms achive the optimal competitiveness of $1 - \frac{1}{e} \approx 0.632$. A natural question is to ask how to break this barrier. This is possible with a relatively small model change.

Model 1 (Random Arrival)**.** *The random arrival model is like the Semi-Online model, but has one crucial difference - vertices arrive in a random order instead of an adversarial one*

In this section we will prove that both Ranking and RGA perform strictly better in the Random Arrival model than in the Semi-Online model

Theorem 3. *In the random arrival model, RGA(Algorithm 3) has a competitiveness of* $1 - \frac{1}{e}$

Theorem 4. *In the random arrival model, Ranking has a competitiveness of at least* 0.696

4.1 RGA

We will present a proof of Theorem 3 by [GM08] which goes in three phases, first proving properties of RGA, then more 3 complex lammas and finally using those to prove the theorem with the help of linear programming

For ease of argumentation we will describe randomness of the algorithm not as an ability to make a truly random choice, but as seeded randomness, meaning that a deterministic algorithm is given an arbitrary number at initialization which can be used to make random decisions and is unknown to the adversary. The latter description of randomness is identical to the former in practice but it makes discussing runs on inputs easier easier. When runs are compared they can be assumed to have the same random seed.

The proof is in three parts.

4.1.1 basic properties

We will begin by proving several properties of RGA.

Lemma 4.1.1 (Prefix)**.** $\forall \sigma_1, \sigma_2 \in \text{Perm}([n]), t \in [n] | (\forall 0 \leq k \leq t | \sigma_1(k) = \sigma_2(k)) \implies$ *runs of RGA on* σ_1 *and* σ_2 *are identical*

Proof. This lemma is relatively trivial. RGA doesn't have access to σ so, while both permutations are identical the runs will be identical as well. □

Let Ω_p be a set of permutations where all vertices but v_p have fixed positions, $\sigma_k \in \Omega_p$ be the permutation where x_p is on position k, $G_s(t)$ be the set of offline vertices matched in time $1, \ldots, t$ by RGA on σ_s

Lemma 4.1.2 (Monotonicity)**.** $\forall 1 \leq s < m \leq n$*:*
- $\forall 1 \leq t \leq s-1$ $G_m(t) = G_s(t)$
- $\forall s-1 \leq t \leq m-1$ $G_m(t) \subseteq G_s(t+1)$

Another way of phrasing this lemma is to say that, if a vertex is moved up in the permutation then all offline vertices matched previously by time t are expected to be matched by time $t+1$

Proof. The first part is a direct consequence of Lemma 4.1.1. Proof of the second part will be by induction over time t, starting with $s-1$ which is directly implied by the first part.
Asumming the lemma holds at time k, $G_m(k) \subseteq G_s(k+1)$. Notice that $\sigma_m(k+1) = \sigma_s(k+2)$. Denote $v := v_{\sigma_m(k+1)}$, matched to u on RGA run on σ_m. Suppose $u \notin G_s(k+2)$. Then u was availible when RGA was matching v on σ_s, but v was instead matched to $u' \notin G_s(k+1)$. By induction hypothesis $u' \notin G_m(k)$ implying u, u' availible at $t = k+1$ in run σ_m, RGA should have chosen u' over u, contradiction implying $u \in G_s(k+2)$ implying step condition for $t = k+1$ □

As context for the last lemma, we need to establish the following: what it means to miss a vertex, what is a good match and what is a bad match.

A vertex is considered missed if it remains unmatched by the end of the run

$$\text{miss}_\sigma(s,p) = \begin{cases} 1 & \sigma(s) = p \wedge \ v_p \text{ unmatched at the end of RGA run on } \sigma \\ 0 & \text{else} \end{cases}$$

if v_p was missed then u_p must have been matched to some $v_{p'}$ and $v_{p'}$ arrived before v_p. We define this match (between u_p and $v_{p'}$) as a bad match since in the end it caused v_p to be missed

$$\text{bad}_\sigma(s,q) = \begin{cases} 1 & u_q \text{ is matched to } v_{\sigma(s)} \wedge \sigma^-1(q) > s \\ 0 & \text{else} \end{cases}$$

in contrast, a good match occurs when $v_{p'}$ arrives after v_p. In that situation we have a guarantee that v_p wasn't missed since u_p and v_p are connected

$$\text{good}_\sigma(s,q) = \begin{cases} 1 & u_q \text{ is matched to } v_{\sigma(s)} \wedge \sigma^-1(q) \leq s \\ 0 & \text{else} \end{cases}$$

Lemma 4.1.3 (Partitioning)**.** $\forall p, \Omega_p$*, during a run of RGA on* σ_n*. If* u_p *is matched to* v_t *then*

$$\forall x \in [1,t] : \sum_{1 \leq s \leq t+1} \text{good}_{\sigma_x}(s,p) = 1 \forall x \in [t+1,n] : \quad \text{bad}_{\sigma_x}(t,p) = 1 \tag{4.1}$$

Proof. 1. from Lemma 4.1.2 we know that $\forall x \in [1,t]$, in a run on σ_x is matched to v_k for some $k \in [x, t+1]$ u_p is matched to v_t implying inequality 1 2. from Lemma 4.1.1 we know that $\forall x \in [t+1,n]$, in a run on σ_x u_p is matched to some v_k for $k \in [x, t+1]$ implying inequality 2 □

4.1.2 bad/good/miss properties

The second part is to prove three helper lemmas about miss, bad, good with the basic properties

Lemma 4.1.4. $\forall \sigma \in \text{Perm}([n]), t \in [n]$
$\sum_{1 \leq p \leq n} \text{miss}_\sigma(t,p) + \sum_{1 \leq p \leq n} \text{bad}_\sigma(t,p) + \sum_{1 \leq p \leq n} \text{good}_\sigma(t,p) = 1$

Proof. this lemma is rather trivial since a vertex is either missed or not, and if it is then either $p < p', p > p'$ or $p = p'$ □

Lemma 4.1.5. $\forall t \in [n]$
$\text{miss}_{\sigma_t}(t,p) \leq \sum_{\sigma \in \Omega_p} \sum_{s<t} \frac{\text{bad}_\sigma(s,p)}{n-s}$

Proof. If miss is 0 then the inequality is trivially true since bad and $n - s$ are non-negative.
Assume miss $= 1$ from definition, in run on σ_t v_p that is on position t remains unmatched, which is only possible if u_p is matched to $v_{p'}$ on position t', $t' < t$ using Lemma 4.1.3 we have

$$\begin{aligned}
&\sum_{\sigma\in\Omega_p}\sum_{s<t}\frac{\mathrm{bad}_\sigma(s,p)}{n-s}\\
\geq&\sum_{\sigma\in\Omega_p}\frac{\mathrm{bad}_\sigma(t',p)}{n-t'} && t'<t\\
\geq&\sum_{t'+1\leq s\leq n}\frac{\mathrm{bad}_{\sigma_s}(t',p)}{n-t'}\\
=&1 && \text{Lemma 4.1.3.2}\\
=&\mathrm{miss}_{\sigma_t}(t,p)
\end{aligned}$$

concluding the proof □

Lemma 4.1.6. $\forall t \in [n-1]$: $\sum_{\sigma\in\Omega_p}\sum_{s\leq t}\mathrm{bad}_\sigma(s,p)\frac{s}{n-s} \leq \sum_{\sigma\in\Omega_p}\sum_{s\leq t+1}\mathrm{good}_\sigma(s,p)$

Proof. **Case 1** u_p remains unmatched on a run on σ_n

Let $\sigma_x \in \Omega_p$, if u_p is matched in a run on σ_x hten by Lemma 4.1.1 it has to be matched to $v_{x'}, x' \leq x$ implying $\forall t | \text{bad}_{\sigma_x}(t,p) = 0$ and the lemma proof becomes trivial by non-negativity

Case 2 u_p is matched in a run on σ_n to $v_{t'}$. We divide this case into two following cases

Case 2.1 $t < t'$

From Lemma 4.1.3 and Lemma 4.1.4 we know which variables are 1, and that for every $\sigma \in \Omega_p, s < t', \text{bad}_\sigma(s,p) = 0$, implying LHS is 0 and the Lemma is again trivial

Case 2.2 finally, if $t \geq t'$

$$\sum_{\sigma \in \Omega_p} \sum_{s \leq t} \text{bad}_\sigma(s,p) \frac{s}{n-s}$$

$$= \sum_{\sigma \in \Omega_p} \text{bad}_\sigma(t',p) \frac{t'}{n-t'}$$

$$= t'$$

$$= \sum_{\sigma \Omega_p} \sum_{s \leq t'+1} \text{good}_\sigma(s,p)$$

$$= \sum_{\sigma \Omega_p} \sum_{s \leq t+1} \text{good}_\sigma(s,p)$$

□

4.1.3 final proof

From here we have the tools to prove the Theorem 3

Notice that the total size of the matching is $\sum_s E\left[\sum_p \text{good}_\sigma(s,p)\right] + \mathbb{E}\left[\sum_p \text{bad}_\sigma(s,p)\right]$.

Informally, the three lemmas suggest that if there are many misses, then there must be many good and bad matches, bounding how small the expected matching can be.

We will start by introducing generalized miss/bad/good variables

$$\text{miss}(s) = \mathop{\mathbb{E}}_{\sigma \sim \text{Perm}([n])}\left[\sum_p \text{miss}_\sigma(s,p)\right]$$

$$\text{bad}(s) = \mathop{\mathbb{E}}_{\sigma \sim \text{Perm}([n])}\left[\sum_p \text{bad}_\sigma(s,p)\right]$$

$$\text{good}(s) = \mathop{\mathbb{E}}_{\sigma \sim \text{Perm}([n])}\left[\sum_p \text{good}_\sigma(s,p)\right]$$

By using Lemma 4.1.5 and summing over all partitions Ω_p and all p we receive thee following

$$\forall t : \sum_p \sum_{\sigma \in \Omega} \text{miss}_\sigma(t,p) \leq \sum_p \sum_{\sigma \in \Omega} \sum_{s<t} \frac{\text{bad}_\sigma(s,p)}{n-s}$$

change summation order, divide by $|\Omega|$

$$\forall t : \text{miss}(t) \leq \sum_{s<t} \frac{\text{bad}(s)}{n-s}$$

by doing the same on Lemma 4.1.6 instead we get

$$\forall t : \sum_{s \leq t} \text{bad}(s) \frac{s}{n-s} \leq \sum_{s \leq t+1} \text{good}(s)$$

with these inequalities we can rewrite the lower bound on expected matching size as a solution of a linear program

$$\begin{array}{lll}
\text{minimize} & \sum_t [\text{bad}(t) + \text{good}(t)] & \\
\text{subject to} & \forall t < n \sum_{s \leq t+1} \text{good}(s) - \sum_{s \leq t+1} \text{bad}(s) \frac{s}{n-s} & \geq 0 \\
& \forall t \leq n \text{good}(t) + \text{bad}(t) + \sum_{s<t} \frac{\text{bad}(s)}{n-s} & \geq 1 \\
& \text{good}(t), \text{bad}(t), \text{miss}(t) & \geq 0
\end{array}$$

which can in turn be rewritten with $m_t = \text{good}(t) + \text{bad}(t), v_t = \frac{\text{bad}(t)}{n-t}$ as the following

$$\begin{array}{lll}
\text{minimize} & \sum_t m_t & \\
\text{subject to } \forall t < n & \sum_{s \leq t+1} m_s - n \sum_{s \leq t} v_s & \geq 0 \\
\forall t \leq n & m_t + \sum_{s<t} v_s & \geq 1 \\
& m_t, v_t & \geq 0
\end{array}$$

Dropping the m_{t+1} from $\sum_{s \leq t+1} m_s$ narrows down the solution space, so the optimum of a program with $\sum_{s \leq t} m_s$ will give a lower bound on the expected competitiveness. The following is that program, alongside it's dual.

$$\begin{array}{lll} \text{minimize} & \sum_t m_t & \\ \text{subject to } \forall t < n & \sum_{s \leq t} m_s - n \sum_{s \leq t} v_s \geq 0 & \\ \forall t \leq n & m_t + \sum_{s < t} v_s \geq 1 & \\ & m_t, v_t \geq 0 & \end{array} \qquad \begin{array}{lll} \text{maximize} & \sum_i \alpha_i & \\ \text{subject to} & \forall t < n \sum_{s > t} \alpha_s - n \sum_{s \geq t} \beta_s & \leq 0 \\ & \forall t \leq n \ \alpha_t + \sum_{s \geq t} \beta_s & \leq 1 \\ & \alpha_t, \beta_t & \geq 0 \end{array}$$

Notice that both primal and dual constraints are satisfied when all inequalities are tight, which in turn implies the optimality of that solution by weak duality.

$$m_t = \alpha_{n+1-t} = (1 - \frac{1}{n})^{i-1} v_t = \beta_{n+1-t} = \frac{1}{n}(1 - \frac{1}{n})^{i-1}$$

with the objective being $\sum m_t \approx n(1 - \frac{1}{e})$, concluding the proof

4.2 Ranking

In this scetion we will prove Theorem 4 following an approach by [MY11]

The approach is to

1. prove two inequalities, which are properties of the Ranking algorithm in the Random Arrival model
2. use these to define a family of exponential size linear programs expLP(n, k)
3. relax it to a family of polynomial size linear programs PolyLP(n)
4. derive a family of Strongly Factor-Revealing linear program PolyLP$'(n)$
5. use computing machines to solve some linear programs from PolyLP$'(n)$ to obtain a lower bound

Do note that the last step is computational and does not prove an exact competitiveness ratio. Indeed, this approach merely shows that the ratio is at least 0.696, and other analysis show it can be no larger than 0.727. The exact competitiveness ratio is currently unknown.

4.2.1 Ranking properties

For the purposes of this section we fix a graph $G = (V, U, E)$
$x(\sigma, \pi, u, v) = 1$ iff $(u, v) \in \text{Ranking}(G, \sigma, \pi)$
$m^\star(u, v) = 1$ iff $M \star (u) = v$.
We will slightly abuse notation and tread vertices as integers in $[n]$, this way for $u \in U$, $\sigma(u)$ is the vertex with rank u. The same occurs for $v \in V$ and arrival time.

Lemma 4.2.1 (Dominance)**.** $\forall \sigma \in \text{Perm}(U), \pi \in \text{Perm}(V), u \in U, v \in V$

$$\sum_{1 \le u' \le u} x(\sigma, \pi, u', v) + \sum_{1 \le v' \le v-1} x(\sigma, \pi, u, v') \ge m^\star(\sigma(u), \pi(v)) \tag{4.2}$$

Proof. This is an observation made before in the proof of Lemma 3.1.2 $(\sigma(u), \pi(v)) \in M^\star$. When v arrives at time $\pi(v) = t$, if it is matched then it must be matched either to u or to u' with a lower rank. The equation follows directly from the observation □

Let $\text{geq}(u_i, u_j) = 1$ iff $u_i \ge u_j$

Lemma 4.2.2 (Monotonicity)**.** $\forall \sigma \in \text{Perm}(U), \pi \in \text{Perm}(V), u_{\text{new}}, u_{\text{old}}, u \in U, v \in V$ *such that* $l_{\text{new}} < l_{\text{old}}, l < l_{\text{old}}$*, Let* σ' *be* σ *where* u_{new} *and* u_{old} *swap places.*

$$\sum_{1 \le u' \le u + \text{geq}(u, u_{\text{new}})} (\sigma', \pi, u', v) \ge \sum_{1 \le u' \le u} x(\sigma, \pi, u', v) \tag{4.3}$$

Proof. Again, this is a claim that is similar to the one in Lemma 3.1.3. The difference between $\text{Ranking}(G, \sigma, \pi)$ and $\text{Ranking}(G, \sigma', \pi)$ is a single alternating path starting from $\sigma(u_{\text{old}})$, implying that if v was matched to some offline vertex with rank at most u in $\text{Ranking}(G, \sigma, \pi)$ then in $\text{Ranking}(G, \sigma', \pi)$ it is matched an offline vertex with rank at most $u + \text{geq}(u, u_{\text{new}})$, which directly implies the inequality □

We can also express a symmetrical equation for π'

$$\sum_{1 \le v' \le v + \text{geq}(v, v_{\text{new}})} (\sigma, \pi', u', v) \ge \sum_{1 \le v' \le v} x(\sigma, \pi, u, v') \tag{4.4}$$

the proof is symmetrical, so we will ommit it

4.2.2 formulating $\text{expLP}(n, k)$

for the purposes of creating $\text{expLP}(n, k)$ for a graph G with n vertices on both sides and k edges, for every $\sigma, \pi, u, v \in \text{Perm}(U) \times \text{Perm}(V) \times U \times V$, $x(\sigma, \pi, u, v)$ is a separate variable. We create a linear program that enforces the properties of x as follows

$$\begin{aligned}
\text{minimize} & \frac{1}{(n!)^2 k} \sum_{\sigma,\pi,u,v} x(\sigma,\pi,u,v) \\
\text{subject to} & \textit{Equation } 4.2 \\
& \textit{Equation } 4.3 \\
& \textit{Equation } 4.4 \\
& x(\sigma,\pi,u,v) \geq 0 \\
& x(\sigma,\pi,u,v) \leq 1
\end{aligned}$$

Lemma 4.2.3. *The competitiveness ratio of Ranking in the Random Arrival model is lower bounded by* $\inf_{n\in N} \inf_{l} \text{expLP}(n,k)$ *for any infinite subset of natural numbers* N

Proof. Let $n \in \mathbb{N}, n' \in \{x \in N | x > n\}$.
Suppose we want to run Ranking on a $n \times n$ graph G. By adding isolated vertices we can change it into a $n' \times n'$ graph with the identical run to that on G when ignoring the isolated vertices
As shown in the lammas in the previous section, a valuation of x is a feasible solution to $\text{expLP}(n,k)$. The value of the objective function is the average over all σ and π divided by n, making it the performance of Ranking on that graph. Note that the linear program is a relaxation, and so provides a lower bound on the competitiveness ration □

4.2.3 relaxation to $\text{PolyLP}(n)$

This method relies in the final step on using LP solvers to obtain a bound rather then proving it analytically. For this reason the exponential size of $\text{expLP}(n,k)$ relative to n is unfortunate as it will make it practically impossible to complete this omputation in a reasonable time. For this reason we will create a family of polynomial size linear programs which in turn provide a good lower bound on the optimal values of the objective function of $\text{expLP}(n,k)$

We assume WLOG that the optimal matching $M^\star = \{(i,i) | i \in [n]\}$

Let $x_1(u,v,p)$ be the probability over random $\sigma,\pi \in \text{Perm}(U) \times \text{Perm}(V)$ that offline agent of rank u is matched to an online agent at time v by Ranking and the offline agent at rank p is matched to the online agent arriving at time v in $M^\star$

Let $x_2(u,v,q)$ be the probability over random $\sigma,\pi \in \text{Perm}(U) \times \text{Perm}(V)$ that offline agent of rank u is matched to an online agent at time v by Ranking and that the online agent arriving at time q will be matched to the offline agent at rank l in $M^\star$

using $\text{eq}(x,y) = 1$ iff $x = y \wedge x, y < k$ where k is the amount of edges, we can define these variables

$$x_1(u,v,p) = \mathop{\mathbb{E}}_{\sigma,\pi}[\mathrm{eq}(\sigma(p),\pi(v))x(\sigma,\pi,u,v)] x_2(u,v,p) = \mathop{\mathbb{E}}_{\sigma,\pi}[\mathrm{eq}(\sigma(u),\pi(q))x(\sigma,\pi,u,v)]$$

We introduce partial sum variables

$$\forall i \in \{1,2\} y_i(u,v,p) = \begin{cases} 0 & \text{if } u = 0 \vee v = 0 \\ \sum\limits_{1 \le u' \le u} x_i(u',r,p) & \text{else} \end{cases}$$

Now we derive the new inequalities for PolyLP(n) multiplying both sides of Equation 4.2 by $\mathrm{eq}(\sigma(u)m\pi(r))$ gives

$$\sum_{1 \le u' \le u} \mathrm{eq}(\sigma(u),\pi(r))x(\sigma,\pi,u',v) + \sum_{1 \le v' \le v} \mathrm{eq}(\sigma(u),\pi(r))x(\sigma,\pi,u,v') \ge \mathrm{eq}(\sigma(u),\pi(v))$$

and taking expectation over ranom σ, π results in

$$y_i(u,v,u) + y_2(v-1,u,v) \ge \frac{k}{n^2} \tag{4.5}$$

We multiply Equation 4.3 similarly, for a fixed p let $u_{\mathrm{old}} = n$ and again take expectation over random σ, π. RHS is $y_1(u,v,p)$.

Notice that σ' is uniformly random, $\sigma(p) = \pi(v)$ iff $\sigma''(p') = \pi(r)$ where σ'' which σ with u_{new}, n swapped and

$$p' = \begin{cases} p & p < u_{\mathrm{new}} \\ p+1 & u_{\mathrm{new}} \le p < n \\ u_{\mathrm{new}} & p = n \end{cases}$$

from which we get the following equation

$$y_1(u + \mathrm{geq}(u, u_{\mathrm{new}}), v, p') \ge y_1(l,r,p) \tag{4.6}$$

and symmetrically from *Equation* 4.4

$$y_2(v + \mathrm{geq}(v, v_{\mathrm{new}}), u, q') \ge y_1(l,r,q) \tag{4.7}$$

with q' being defined symmetrically to p'. Finally we add

$$\sum_p x_1(u,v,p) = \sum_q x_2(v,u,q)$$

as both sides are equal to $\frac{n}{k}\mathop{\mathbb{E}}_{\sigma,\pi}[x(\sigma,\pi,u,v)]$. Together, these inequalities also imply non-negativity and with y_i replaced with their definitions on x_i these inequalities form the LP constraints.

The objective function is defined as $\frac{1}{2k}\left(\sum_{u,v,p} x_1(u,v,p) + \sum_{u,v,q} x_2(v,u,q)\right)$, equivalent to the previous objective.

Before continuing to the last step we will implement the following simplifications to PolyLP(n).

- $x_1 = x_2$ since for a feasible x_1, x_2 $x_1' = x_2' = \frac{x_1+x_2}{2}$ is also feasible and this reduces the amount of variables by 2
- WLOG $k = n$ since the fact that a solution for k can be rescaled to k' makes k inconsequential
- and several other which are more technical

which finally results in

$$\begin{aligned}
&\text{Minimize} \frac{1}{n}\sum_{u,v,p\in[n]} x(u,v,p)\\
&\text{subject to} y(u,v,u) + y(v-1u,v) \geq \frac{1}{n}\\
&\qquad y(u+1,v,p+1) \geq y(u,v,p) && \forall p \leq l < n\\
&\qquad y(u,v,p) = y(u,v,u+1) && \forall u < p\\
&\qquad y(u+1,r,p) \geq y(u,v,u+1) && \forall p \leq u < n\\
&\qquad \sum_p x(u,v,p) = \sum_p x(v,u,p)
\end{aligned}$$

4.2.4 deriving PolyLP$'(n)$

For a maximization algorithm a family of Linear Programs is strongly factor-revealing iff the solution of any linear program in it is a lower bound on the approximation ratio of the algorithm. It is stronger than the more common factor revealing family for which the infimum of the optimum solutions is the lower bound.

PolyLP(n) is factor-revealing, in this section we will transform it into a strongly factor-revealing family PolyLP$'(n)$. To do this, we will replace the first constraint with

$$y(u,v,u) + y(v,u,p) \geq \frac{1}{n} \tag{4.8}$$

Lemma 4.2.4. *the family of linear programs* PolyLP$'(n)$ *is strongly factor-revealing for Ranking with random arrivals*

Proof. Take eny integer $m > 0$. For any instance of size $n = km, k \in \mathbb{Z}$. We will prove that the competitiveness ratio of Ranking is lower-bounded by PolyLP$'(m)$. We know that $\inf_d \text{PolyLP}(md)$ is greater or equal to the desired competitiveness ratio. We will take an optimal solution (x, y) of PolyLP(md) and project it to a feasible solution of PolyLP$'(m)$ as follows

Definition 4.2.1. *For any $i, j, k \in [m]$ define $f(i) = \{d(i-1)+1, \ldots, d*i\}, f(i,j) = f(i) \times f(j), f(i,j,k) = f(i) \times f(j) \times f(k)$ ($\times$ denotes Cartesian product)*
For any $u', v', p' \in [m]$ define $x'(u', v', p') = \frac{1}{d} \sum_{(u,v,p) \in f(u',v',p')} x(u,v,p)$,$y'(u', v', p') = \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, p)$

We will show what x', y' for PolyLP$'(m)$ has the same objective value as x, y for PolyLP(m)

$$\begin{aligned}
&\frac{1}{m} \sum_{u',v',p' \in [m]} x'(u', v', p') \\
=&\frac{1}{m} \sum_{u',v',p' \in [m]} \frac{1}{d} \sum_{(u,v,p) \in f(u',v',p')} x(u,v,p) \\
=&\frac{1}{n} \sum_{u,v,p \in [n]} x(u,v,p) \\
=&\text{PolyLP}(n)
\end{aligned}$$

Next, we show that x', y' is feasible in PolyLP(n)
The fifth constraint of PolyLP$'(n)$ can be obtained by summing the fifth constraint of PolyLP(n) over $(u, v) \in f(u', v')$
The sixth constraint of PolyLP$'(n)$ can be obtained by summing the sixth constraint of PolyLP(n) over $(v, p) \in f(v', p'), u = u'd$
The second constraint of PolyLP$'(n)$ can be obtained as follows: for $p' \leq u' < m$

$$\begin{aligned}
y'(u'+1, v', p'+1) &= \frac{1}{d} \sum_{(r,p) \in f(v',p')} y(u'd + d, v, p + d) \\
&\geq \frac{1}{d} \sum_{(r,p) \in f(v',p')} y(u'd, v, p) \\
&= y'(u', v', p')
\end{aligned}$$

where the inequality is the sesult of chaining multiple $y(u'd + j + 1, r, p + j + 1) \geq y(u'd + j, v, p + j)$ inequalities
The third constraint of PolyLP$'(n)$ can be obtained as follows: for $u' < p' \leq n$, $r' \leq n$

$$\begin{aligned}
y'(u', v', p') &= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, p) \\
&= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, r, u'd + 1) \\
&= \frac{1}{d} \sum_{(v,p) \in f(v',u'+1)} y(u'd, v, p) \\
&= y'(u', v', u' + 1)
\end{aligned}$$

The fourth constraint of PolyLP$'(n)$ can be obtained as follows: for $p' \leq u' < n, v' \leq n$

With Lemma 4.2.4 proven, the theoretical framework of this proof is complete and we cite the computational result. In [MY11], appendix B state that the optimum solution for PolyLP$'(50)$ is 0.696150. We also note that these results were obtained using CPLEX software.

Chapter 5

Stochastic Arrivals

Definition 5.0.1 (iid Model)**.** *In the iid model the algorithm always knows a set of offliene vertices U, but there is no determined set of online vertices V. Instead, there is a set V' of vertex 'archetypes' consisting of a neighborhood $N \subseteq U$ and a probability p. When a new vertex v arrives, for each $(N, p) \in V'$ v has of p to have neighborhood N. The archetype set V' is known to the algorithm*

We note two things here:

1. obviously, $\sum_{(N,p)\in V'} p = 1$
2. This model can no longer be described as a game against an adversary since the adversary has been stripped of all agency

We will present the proof of two theorems in thi chapter:

Theorem 5. *No algorithm has a competitiveness of $1 - \varepsilon$ for an arbitrary ε in the iid model*

meaning that there is no perfect algorithm, and

Theorem 6. *There is al algorithm with a competitiveness of at least ≈ 0.729*

which is substantially better than the 0.696 guaranteed under random arrivals, though not exceeding it's upper bound of 0.727

5.1 Competitiveness Limit

In this section we will follow a proof from (beating 1-1/e). Consider a 6-Cycle defined as follows:

Definition 5.1.1. $U = \{1, 2, 3\}, V' = \{x = (\{1, 2\}, \frac{1}{3}), y = (\{1, 3\}, \frac{1}{3}), z = (\{2, 3\}, \frac{1}{3})\}$

If x arrives first, if it is matched to 1 and then two y vertices y_1, y_2 arrive, the second will be unmatchable. In this scenario there is a perfect matching $M\star = \{(2, x), (1, y_1), (3, y_2)\}$

This argument is symmetric, for any first arrival v matched to u there is an archetype v' that forces this scenario.
The probability of this scenario is therefore the probability of (v', v') arriving after v, which is $\frac{1}{9}$. Assuming that in all other scenarios the algorithm find the perfect matching, competitiveness is ar most $\frac{1}{9}\frac{2}{3} + \frac{8}{9}1 = \frac{26}{27}$.

Therefore, the theorem is proven and the competitiveness of any algorithm in the iid model is bounded by ≈ 0.962

5.2 0.729 Algorithm

Chapter 6

Delay Model

In the Delay model the Algorithm ca wait a certain amount of time before matching a vertex

Theorem 7. *For delay no greater than $n/2$ any deterministic online algorithm has competitiveness of* 0.5

Proof. Consider the following graph G (figure above): There are $k = n/2$ groups $\{Z_i | i \in [k]\}$. of two offline vertices
First, k vertices z_i^D arrive, each connected to both vertices of Z_i.
As z_k^D arrives the cache is full, and z_1^D has to be matched. Let the offline vertex it was matched to be z_1^B. Next, z_1^C arrives, only connected to z_1^B, which is unmatchable. Let z_1^A be the unmatched offline vertex.
After z_1^A has arrived z_2^D has to be matched. The procedure above is repeated until z_k^C has arrived.
The perfect matching of G connects all z_i^A, z_i^D and z_i^B, z_i^C pairs, while the algorithm connects only z_i^B, z_i^D pairs, making the competitiveness 0.5. The final graph is illustrated in the figure below. □

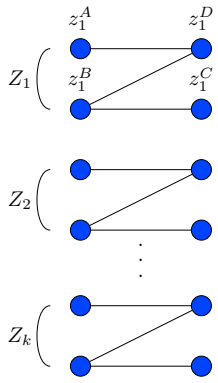


Figure 6.1: Worst case scenario for a deterministic cache algorithm

Chapter 7

Final Degree Model

In the Final Degree Model, The algoritm has, at all times, the knowledge of the what the degree of all offline edges is in the final graph. This allows an algorithm to know how many opportunities there will be for a vertex to be matched and let it focus on the most urgent vertex. A natural expression of this idea is the effective degree algorithm.

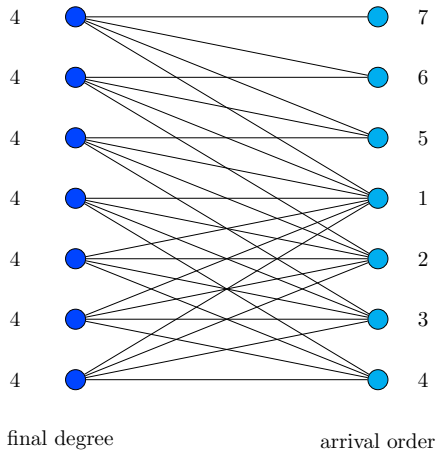
Algorithm 4 Effective Degree

on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$
on Arrival of vertex v: N are unmatched offline neighbors of v. If $N \neq \emptyset$, match v to $\underset{u \in N}{\operatorname{argmin}}\ d_u$, $\underset{u \in N}{\forall} d_u - = 1$

Despite this algorithm seeming promising, it is only marginally more effective than a naive deterministic approach. We will show that in this model no deterministic algorithm has asymptocic competitiveness greater then 0.5

Theorem 8. *In the Final Degree model no deterministic algorithm has a competitiveness higher than* 0.5

Proof. Consider the graph G_{deg} which has $n = 2k + 1$ offline vertices with final degree $k + 1$ and is formed as follows:
$k + 1$ vertices arrive, all connected to all unmatched offline vertices.
Notice that at this point $k + 1$ vertices are matched and the remaining k have degree $k + 1$ in the current graph, meaning that they cannot be matched. All that remains is to fulfill the degree promise. This is easy as the degrees of the matched vertices are $1, 2, \ldots, k, k + 1$ meaning that all that is required is for k vertices connected to all offline vertices with unfulfilled degree promise to arrive. The size of the matching found is $k + 1$ while the graph has a perfect matching of size $n = 2k + 1$ which is achived when the k vertices matched by the graph are instead matched to the vertices which fulfilled their degree promise while the remaining $k + 1$ vertices are matched among themselves since they form a biclique $K_{k+1,k+1}$ □



We will remark here that this model is a relaxation of the Semi-Online model, so competitiveness here isn't lower

7.1 Randomized Augmentation

As shown above, the information of a graph's final degree isn't helpful for deterministic algorithms. However, there is a good reason to believe that this might not be the case for rndomized algorithms
Here, the fact that the competitiveness is only 0.5 asymptotically and in actuality is $\frac{k+1}{2k+1}$ is key.

Lemma 7.1.1. *If all vertices have different degrees then the algorithm finds a perfect matching*

Proof. Let $G_n = (V, U, E)$ be a $n \times n$ bipartite graph with with all vertices in U having different degrees. WLOG G has a perfect matching, implying that the degrees of $u \in U$ are $1, \ldots, n$. Let $u_k := u \in U | deg(u) = k$
To force the algorithm to leave n_k behind, k online vertices need to arrive, each connected to a different $u'_i \in \{u_i | i < k\}$. The problem with this is that the size of U' is $k - 1$, maiing sich a thing impossible □

A much stronger realization can be gleamed from this reasoning to force a deterministic algorithm to leave a vertex u behind, and for it to be possible for a randomized algorithm to leave that vertex behind, there must be $\deg(u)$ vertices v such that $\deg(u) \geq \deg(v)$, which moreover won't be left behind.

This line of reasoning leads directly to the graph $G_{\deg}$ used in the proof of Theorem 8. However, this also leads to a problem for an adversary. That graph is rather dense, which isn't a situation where randomized graphs perform poorly. Indeed, despite RGA (Algorithm 3) having an expected competitiveness of $\frac{n}{2} + O(\log n)$, it is expected to find a matching of size $n(1 - \frac{1}{e})$ on a graph which is actually a subgraph of the degree countergraph edgewise, meaning it would work at least as well here

Algorithm 5 Randomized Effective Degree

on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$
on Arrival of vertex v: N are unmatched offline neighbors of v. If $N \neq \emptyset$, let $d = \min_{u \in N} d_u$. $v = \text{random}(\{u \in N | \deg(v) = d\})$. Match v to u, $\forall_{u \in N} d_u - = 1$

Lemma 7.1.2. *On $G_{\deg}$ from Theorem 7 Algorithm 5 has an expected performance of no less than ≈ 0.75*

Proof. We can divide the arriving online vertices into two phases: first being when the first $k+1$ vertices that make the upper k offline vertices unmatchable in the deterministic scenario, and the second phase when latter k online vertices arrive and are unmatchable.
In the first phase the arriving vertices are always matchable since they are connected to all offline vertices
In the second phase arriving vertices are connected to incrementally smaller parts of the upper k vertices
The lower $k+1$ offline and online vertices form a biclique, meaning that all matches made between the first $k+1$ online vertices and the lower $k+1$ offline vertices do not make any vertex unmatchable. Since matches are chosen randomly, we expect half of the online agends from the first phase to be matched there
In the second phase k vertices arrive connected to incrementally smaller parts of the upper k offline vertices. Using an argument symmetrical to the one in Lemma 7.1.1 we know that the matching here is perfect
In total - all online vertices in the first phase have been matched and an expected half of the vertices in the second phase has ben matched, resulting in an expected performance of 0.75 □

Note here that we have conducted Monte Carlo experiments with this exact algorithm and graph for k up to 50 and in over 1 million simulations the algorithm found a matching of average size $n-2$ suggesting that the actual performance is much higher here

¡If it is possible to prove that different degrees always result in the algorithm kaing better decisions, then Algorithm 5 has a competitiveness of 0.75 since minimizing it's ability to make correct decisions, the final graph must be a series of disconnected G_{deg} of various sizes¿

7.2 Performance on random graphs

as shown before, despite it's potential the Final Degree model doesn't allow for a better competitiveness. However, unlike in the Semi-Online model, the class of graphs that can force a pessimistic matching of size $\frac{n}{2}$ is very limited. For this reason, it is alwo worth analyzing performance in a much more forgiving, and realistic, enviorenment.

Definition 7.2.1. *The CLV-B model is a bipartite adaptation of the CLV model for general graphs introduced by Fan Chung, Linyuan Lu, and Van Vu. It is defined by two vectors, $\vec{p} \in [0,1]^n, \vec{q} \in [0,1]^m$.An edge between $u_j \in U, v_i \in V$ appears with probability $\vec{p}_j\vec{q}_i$. We assume WLOG that elements of $\vec{p}$ are in non-increasing order.*
We denote the random graph of vectors $\vec{p}, \vec{q}$ $I_{\vec{p},\vec{q}}$

Do note that in this model the matching can no longer be expressed as a game since there are no choices for an adversary to make and $\vec{u}, \vec{v}$ are merely

descriptors of a class of random graphs
Since the model is probabilistic and there is no final graph, the RED algorithm, when adapted to this setting, is as follows

Algorithm 6 RED-CLV

on Initialization: for each offline vertex u_i let $d_{u_i} = \sum_{v=\in\vec{v_i}} v\vec{u}_i$

on Arrival of vertex v: match to $\underset{u\in U,\text{unmatched}}{\operatorname{argmin}} d_u$

Theorem 9. *Algorithm 5 (RED) is optimal in CLV-B model, meaning for any algorithm* $\mathrm{A}, t \in \mathbb{Z}_+, \vec{p} \in [0,1]^n, \vec{q} \in [0,1]^m$ $\Pr[\mathrm{A}(I_{\vec{p},\vec{q}}) > t] \leq \Pr[\mathrm{RED}(I_{\vec{p},\vec{q}}) > t]$

the proof is using an approach used by [CI21]

Lemma 7.2.1.

Lemma 7.2.2.

Bibliography

[KVV90] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. "An optimal algorithm for on-line bipartite matching". In: *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*. STOC '90. Baltimore, Maryland, USA: Association for Computing Machinery, 1990, pp. 352–358. ISBN: 0897913612. DOI: 10.1145/100216.100262. URL: https://doi.org/10.1145/100216.100262.

[BM08] Benjamin Birnbaum and Claire Mathieu. "On-line bipartite matching made simple". In: *SIGACT News* 39.1 (Mar. 2008), pp. 80–87. ISSN: 0163-5700. DOI: 10.1145/1360443.1360462. URL: https://doi.org/10.1145/1360443.1360462.

[GM08] Gagan Goel and Aranyak Mehta. "Online budgeted matching in random input models with applications to Adwords". In: *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '08. San Francisco, California: Society for Industrial and Applied Mathematics, 2008, pp. 982–991.

[MY11] Mohammad Mahdian and Qiqi Yan. "Online bipartite matching with random arrivals: an approach based on strongly factor-revealing LPs". In: *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*. STOC '11. San Jose, California, USA: Association for Computing Machinery, 2011, pp. 597–606. ISBN: 9781450306911. DOI: 10.1145/1993636.1993716. URL: https://doi.org/10.1145/1993636.1993716.

[CI21] Justin Y. Chen and Piotr Indyk. "Online Bipartite Matching with Predicted Degrees". In: *CoRR* abs/2110.11439 (2021). arXiv: 2110.11439. URL: https://arxiv.org/abs/2110.11439.